B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institute, Affiliated to VTU)

Bull Temple Road, Basavanagudi, Bengaluru – 560019

A Project Work-2 [23IS7PWPW2] Report on

Comparative Evaluation of Machine Learning and Neural Networks for Lead Conversion Prediction in Real-Estate and Construction Leads

Submitted in partial fulfilment of the requirements for the award of degree

BACHELOR OF ENGINEERING

IN

INFORMATION SCIENCE AND ENGINEERING

By

Anant Ganapati Bhat (**1BM24IS400**)

Bandi Venkata Sai Vishal (**1BM23IS052**)

Bharath U (**1BM23IS053**)

Hritik H (**1BM23IS305**)

Under the guidance of

Ms. Manjula S

ASSISTANT PROFESSOR

Department of Information Science and Engineering

2026–2027



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institute, Affiliated to VTU)

Bull Temple Road, Basavanagudi, Bengaluru – 560019

Department of Information Science and Engineering

C E R T I F I C A T E

This is to certify that the Project Work-2 [23IS7PWPW2] entitled **"Comparative Evaluation of Machine Learning and Neural Networks for Lead Conversion Prediction in Real-Estate and Construction Leads"** is a bona-fide work carried out by **Anant Ganapati Bhat (1BM24IS400)**, **Bandi Venkata Sai Vishal (1BM23IS052)**, **Bharath U (1BM23IS053)** and **Hritik H (1BM23IS305)** in partial fulfilment of the requirements for the award of the degree of Bachelor of Engineering in Information Science and Engineering from Visvesvaraya Technological University, Belagavi, during the academic year 2026–2027.

It is certified that the corrections and suggestions indicated during the internal assessments have been incorporated in the report deposited in the departmental library. The project report has been approved as it satisfies the academic requirements prescribed for the Bachelor of Engineering degree.

Ms. Manjula S	**Dr. M. K. Nalini**	**Dr. Bheemsha Arya**
Assistant Professor	Professor and HOD	Principal

Examiners

Name of the Examiner	Signature of the Examiner
1. _____	_____
2. _____	_____

DECLARATION

We, **Anant Ganapati Bhat (1BM24IS400)**, **Bandi Venkata Sai Vishal (1BM23IS052)**, **Bharath U (1BM23IS053)** and **Hritik H (1BM23IS305)**, students of Bachelor of Engineering in Information Science and Engineering at B.M.S. College of Engineering, Bengaluru, hereby declare that the Project Work-2 [23IS7PWPW2] entitled **"Comparative Evaluation of Machine Learning and Neural Networks for Lead Conversion Prediction in Real-Estate and Construction Leads"** is an authentic work carried out by us under the supervision and guidance of **Ms. Manjula S**, **Assistant Professor**, Department of Information Science and Engineering, B.M.S. College of Engineering.

We further declare that this report has not been submitted, in whole or in part, to any other university or institution for the award of any other degree or diploma.

Place: Bengaluru

Date: _____

Name	**USN**	**Signature**
Anant Ganapati Bhat	1BM24IS400	
Bandi Venkata Sai Vishal	1BM23IS052	
Bharath U	1BM23IS053	
Hritik H	1BM23IS305	

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of this Capstone Project Phase-2 would be incomplete without the mention of the people who made it possible through constant guidance and encouragement.

We would take this opportunity to express our heart-felt gratitude to **Dr. B. S. Ragini Narayan**, Chairperson, Donor Trustee, Member Secretary & Chairperson, BMSET. **Dr. P. Dayananda Pai**, Member Life Trustee, BMSET and **Dr. Bheemsha Arya**, Principal, B.M.S. College of Engineering for providing the necessary infrastructure to complete this Capstone Project-2.

We wish to express our deepest gratitude and thanks to **Dr. M.K. Nalini**, Head of the Department, Information Science and Engineering and the Project Coordinators **Dr.Mahalakshmi B S** and **Prof. Harini S** for their constant support.

We wish to express sincere thanks to our guide **Ms. Manjula S**, **Assistant Professor**, Department of Information Science and Engineering for helping us throughout and guiding us from time to time.

A warm thanks to all the faculty of Department of Information Science and Engineering, who have helped us with their views and encouraging ideas.

Anant Ganapati Bhat (1BM24IS400)
Bandi Venkata Sai Vishal (1BM23IS052)
Bharath U (1BM23IS053)
Hritik H (1BM23IS305)

ABSTRACT

Lead conversion prediction estimates whether a prospective customer will progress from an inquiry to a successful conversion. In real-estate and construction-related businesses, lead records can include behavioural, preference, budget, and engagement information, and prioritising these leads can support more efficient sales follow-up. This project compares conventional machine-learning classifiers and neural-network models for predicting the binary target `conversion_status`. The workflow represented in the project notebooks includes preprocessing and encoding, a stratified training/test partition, ten initial machine-learning models, hyperparameter tuning, a soft-voting ensemble, class-imbalance experiments using class weights and SMOTE, and three neural-network architectures. Evaluation uses accuracy, precision, recall, F1-score, ROC-AUC, confusion matrices, cross-validation scores, and validation learning curves.

The classification notebook reports 12,000 records, an 80:20 training/test split (9,600 and 2,400 records), and 84 numeric input features after excluding the identifier, split indicator, and target. The positive class represents approximately 27% of records. The highest test accuracy among the evaluated configurations is 0.7792 for the tuned soft-voting ensemble, while tuned XGBoost is selected by the training-side selection score (0.7833) and reports test accuracy 0.7762, precision 0.6412, recall 0.3889, F1-score 0.4841, and ROC-AUC 0.7807. These results exceed the majority-class baseline of approximately 73%, but do not meet the aspirational 92% accuracy target. Experiments that emphasise minority-class recall through resampling or class weighting achieve higher recall but lower accuracy and precision, demonstrating the importance of selecting metrics according to business objectives. The report also discusses feature importance and SHAP interpretation. A validation concern remains: target encoding and the cross-validation design must be audited to ensure that all label-derived transformations are fitted within each validation fold. The results are therefore presented as a comparative experimental study, not as evidence of production readiness.

Keywords: lead conversion, lead scoring, binary classification, machine learning, neural networks, XGBoost, CatBoost, SMOTE, SHAP, real-estate analytics.

Contents

Acknowledgement **3**

Abstract **4**

1 INTRODUCTION **1**

1.1 Overview 1

1.2 Motivation 1

1.3 Objectives 1

1.4 Scope 2

1.5 Existing System 2

1.6 Proposed System 2

2 PROBLEM STATEMENT **4**

2.1 Problem Statement 4

2.2 Motivation 4

2.3 Project Objectives and Success Criteria 4

2.4 Dataset Description 4

3 DETAILED SURVEY **6**

3.1 Machine Learning for Classification 6

3.2 Neural Networks for Tabular Data 6

3.3 Class Imbalance 6

3.4 Model Interpretation 7

3.5 Research Gap and Project Contribution 7

3.6 Survey Summary Table 7

4 SYSTEM REQUIREMENT SPECIFICATION **9**

4.1 Functional Requirements 9

4.2 Non-functional Requirements 9

4.3 Hardware Requirements 10

4.4 Software Requirements 10

5 SYSTEM DESIGN **11**

5.1 System Architecture 11

5.2 Module Design 11

5.3 Detailed Design 11

5.3.1 Data Preprocessing . . . 11
5.3.2 Class Diagram . . . 12
5.3.3 Activity Diagram . . . 12
5.3.4 Use Case Diagram . . . 13
5.3.5 Scenarios . . . 13

6 IMPLEMENTATION 14
6.1 Development Environment and Input . . . 14
6.2 Preprocessing Implementation . . . 14
6.3 Baseline Machine-Learning Models . . . 14
6.4 Hyperparameter Tuning . . . 14
6.5 Soft-Voting Ensemble . . . 15
6.6 Imbalance Handling . . . 15
6.7 Deep-Learning Models . . . 15
6.8 Model Saving and Reproducibility . . . 15

7 RESULTS 16
7.1 Evaluation Metrics . . . 16
7.2 Class Distribution . . . 16
7.3 Baseline Confusion Matrices . . . 17
7.4 Comparison Across Model Families . . . 17
7.5 Summary of All 24 Configurations . . . 19
7.6 Selected Conventional Model . . . 19
7.7 Neural Network Results . . . 21
7.8 Class Weighting and SMOTE . . . 21
7.9 Feature Importance . . . 22
7.10 SHAP Analysis . . . 22
7.11 Accuracy Target and Interpretation . . . 24

8 TESTING 25
8.1 Testing Strategy . . . 25
8.2 Important Validation Risks . . . 26
8.2.1 Target-Encoding Cross-Validation . . . 26
8.2.2 Potential Target Leakage in `lead_score` . . . 26
8.2.3 Test-Set Reuse . . . 27

9 CONCLUSION AND FUTURE ENHANCEMENTS 28
9.1 Conclusion . . . 28
9.2 Future Enhancements . . . 28

Bibliography **29**

List of Tables

2.1 Dataset summary as reported in the classification notebook 5

3.1 Selected literature relevant to the project 7

4.1 Software components used or indicated by the notebooks 10

7.1 Complete comparison of evaluated model configurations 20

8.1 Testing scenarios and expected outcomes 25

List of Figures

1.1 High-level experimental workflow represented in the project notebooks. 3

5.1 Conceptual component diagram for the experimental workflow. 12
5.2 Activity flow for model comparison. 12
5.3 Use-case view of the research workflow. 13

7.1 Target class distribution and positive-class share across the stratified split (notebook output). 17
7.2 Confusion matrices for the initial machine-learning models on the test set. . . . 17
7.3 Model comparison visualisation from the notebook. 18
7.4 Additional model comparison visualisation from the notebook. 19
7.5 Selected-model confusion matrix and ROC curve from the notebook. 21
7.6 Training and validation accuracy/loss curves for the selected neural network. . . 21
7.7 Top feature importances across tree-based models, as generated by the notebook. 22
7.8 SHAP global feature-attribution plots from the notebook. 23
7.9 Second SHAP summary visualisation from the notebook. 24

CHAPTER 1 INTRODUCTION

1.1 Overview

Lead generation is a major component of customer acquisition in real-estate and construction businesses. Leads may arrive through websites, digital advertisements, phone enquiries, property portals, referrals, or project campaigns. Sales teams often have limited time and cannot follow up with every lead at the same intensity. A lead-conversion model uses historical records to estimate the likelihood that a lead will convert, helping teams prioritise outreach and understand patterns associated with successful conversion. Predictive lead scoring is a binary classification problem when the outcome is represented as converted or not converted.

Machine learning offers a way to learn non-linear relationships between lead attributes and conversion outcomes. Logistic regression provides a comparatively interpretable baseline; decision trees and ensemble methods can model interactions; support vector machines and nearest-neighbour methods offer alternative decision boundaries; and neural networks can learn layered representations. The project compares these model families under a shared data partition and a common set of evaluation metrics. The notebook pipeline uses 84 numeric features and evaluates a total of 24 model configurations, including tuned and imbalance-handling variants.

1.2 Motivation

A fixed or intuition-based lead-prioritisation strategy may not use the information contained in previous conversion outcomes consistently. Predictive models can support more systematic prioritisation, but an apparently high accuracy can be misleading when most records belong to the negative class. For this reason, the project examines precision, recall, F1-score, and ROC-AUC in addition to accuracy. Recall matters when missing a likely converter is costly; precision matters when sales follow-up resources are limited. The comparison of class weighting and SMOTE further explores the trade-off between detecting minority-class conversions and maintaining overall accuracy.

1.3 Objectives

The objectives are:

1. Prepare the available lead dataset for supervised binary classification.

2. Compare ten baseline machine-learning algorithms using a common train/test split.

3. Tune selected algorithms using training-only cross-validation.

4. Evaluate a soft-voting ensemble of tuned classifiers.
5. Investigate class imbalance using class weights and SMOTE.
6. Compare multilayer perceptron (MLP), deep MLP, and residual MLP models.
7. Analyse results with confusion matrices, ROC curves, learning curves, feature importance, and SHAP plots.
8. Identify methodological limitations and propose validation steps before deployment.

1.4 Scope

The scope is the offline comparison of the models represented in the two supplied Jupyter notebooks. The report documents the reported dataset size, preprocessing workflow, model configurations, metrics, and figures. It does not claim live integration with a customer relationship management system, deployment to a cloud platform, temporal validation, causal explanation, or commercial impact measurement, because these are not established by the notebooks. The 92% accuracy target is treated as an aspiration, not as a result.

1.5 Existing System

A manual lead management process typically prioritises enquiries using sales experience, simple rules, or recent activity. Such approaches can be useful but may be inconsistent across staff and difficult to evaluate quantitatively. A simple rule-based score may also miss interactions between preferences, budget, engagement, and other features. In a data-driven system, historic outcomes are used to train a model, and evaluation on held-out records estimates how well the model generalises. However, machine-learning scores can still be unreliable if labels are noisy, features leak information from the outcome, or the test set is repeatedly used to select models.

1.6 Proposed System

The proposed experimental workflow starts from the preprocessed numeric dataset and its stored split where available. Identifier, split-bookkeeping, and target columns are excluded from the input matrix. The notebook trains baseline classifiers, tunes selected models using cross-validation on training data, evaluates imbalance strategies, and trains neural networks with a separate validation partition. Predictions are compared using several metrics, and interpretation plots are generated for the selected tree model. Figure 1.1 summarises the intended workflow.

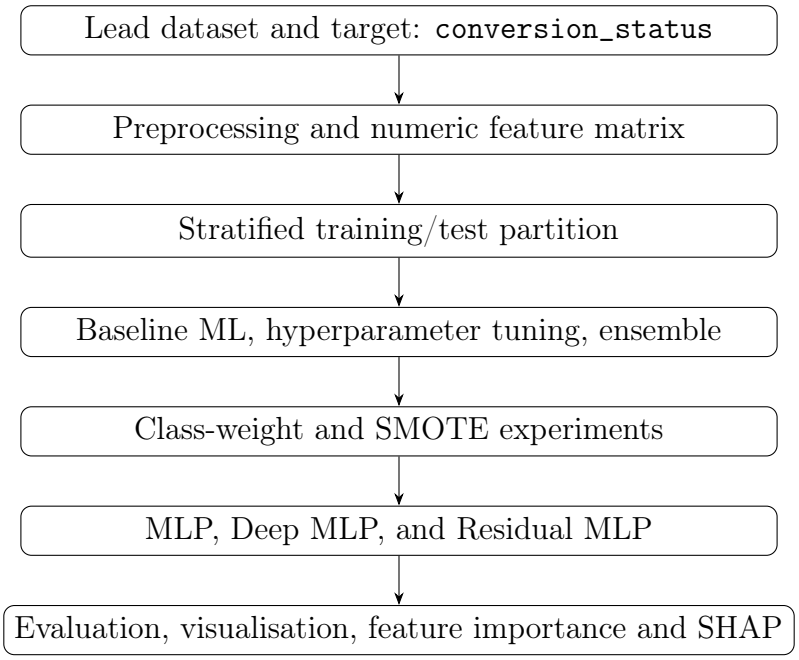


Figure 1.1: High-level experimental workflow represented in the project notebooks.

CHAPTER 2 PROBLEM STATEMENT

2.1 Problem Statement

Given a lead record represented by a feature vector x, predict the binary conversion label $y \in \{0, 1\}$, where $y = 1$ denotes a converted lead and $y = 0$ denotes a non-converted lead. A classifier estimates a decision or score:

$$\hat{y} = f(x), \qquad \hat{p} = P(y = 1 \mid x).$$

The model should provide useful separation between likely converters and non-converters while controlling false positives and false negatives. The project seeks a rigorous comparison of model families and does not assume that any one metric fully captures business value.

2.2 Motivation

The dataset is imbalanced, with approximately 73% negative and 27% positive outcomes. A classifier that predicts every lead as not converted can therefore achieve approximately 73% accuracy while failing to identify any converters. Evaluation must compare models against this baseline and inspect minority-class recall and precision. The most suitable operating threshold may depend on sales capacity, follow-up costs, and the cost of missing a genuine conversion opportunity; those business costs were not provided in the notebooks and should be defined before operational deployment.

2.3 Project Objectives and Success Criteria

The experiment compares the models using a held-out test set and training-side selection scores. The notebook's aspirational target is 92% accuracy. The best observed test accuracy is approximately 77.92%, so the target was not achieved. This outcome should be reported honestly. A useful model can still provide lift over the majority-class baseline, but the degree of practical benefit requires prospective validation and a decision threshold aligned with sales-team capacity.

2.4 Dataset Description

The classification notebook reports 12,000 observations, with 9,600 records in training and 2,400 in testing, and 84 numeric features used for model training. The target is `conversion_status`. The identifier `lead_id`, split bookkeeping field, and target are removed from the feature matrix.

The exact data source, collection dates, consent status, geographic coverage, and licensing terms are not documented in the notebooks; these details should be added from the dataset owner before the final academic submission.

Table 2.1: Dataset summary as reported in the classification notebook

Property	**Reported value**
Prediction target	`conversion_status` (binary)
Total observations	12,000
Training observations	9,600 (80%)
Test observations	2,400 (20%)
Input features	84 numeric features
Positive-class share	Approximately 27%
Negative-class share	Approximately 73%
Identifier handling	`lead_id` excluded from model features
Split handling	Stored split reused where present; stratified split otherwise

CHAPTER 3 DETAILED SURVEY

3.1 Machine Learning for Classification

Supervised classification learns a mapping from input attributes to known labels. Logistic regression estimates a probability through a logistic link and is commonly used as a baseline for binary outcomes. Decision trees divide the feature space into regions; ensemble methods combine multiple trees to reduce variance or improve predictive performance. Random Forest uses bootstrap samples and feature subsampling, while Extra Trees adds further randomisation. Gradient boosting builds an additive sequence of weak learners. XGBoost and LightGBM are efficient gradient-boosting implementations, while CatBoost is designed to handle categorical features effectively in its native workflows. In this project, the model matrix is already numeric, so the comparison evaluates the resulting encoded representation.

Support vector machines seek a separating boundary with a margin and can use kernels for non-linear classification. K-nearest neighbours classifies a point using nearby examples and is sensitive to feature scaling and the distance metric. No single model family is universally best; model performance depends on data characteristics, preprocessing, tuning, and the evaluation protocol.

3.2 Neural Networks for Tabular Data

A multilayer perceptron composes affine transformations and non-linear activation functions. Hidden layers can model complex interactions, while dropout and batch normalisation are often used to regularise training. The project evaluates a basic MLP, a deeper MLP, and a residual MLP. Neural networks can be effective on tabular data, but they are not guaranteed to outperform tree ensembles. The comparison therefore uses the same held-out test set and reports the validation selection score separately from test performance.

3.3 Class Imbalance

When one class is more common than another, a classifier may favour the majority class. Class weighting changes the relative penalty assigned to errors from different classes. SMOTE synthesises minority-class examples from neighbourhood relationships in feature space [4]. These strategies can increase recall, but may reduce precision or accuracy. In this project, class-weighted logistic regression and XGBoost and SMOTE-based variants are compared with the unadjusted models. Resampling should be applied only inside each training fold; applying it before splitting can contaminate evaluation.

3.4 Model Interpretation

Tree feature importance can provide a first indication of which variables the model uses, but impurity-based importance may favour features with particular statistical properties and does not establish causality. SHAP assigns additive feature attributions to model predictions and can be used to inspect global and individual contributions [5]. In this project, SHAP plots are generated from a sample of 500 test records for a selected tree-based model. Interpretation must be cautious where features are correlated or may encode downstream information. In particular, the origin and timing of the `lead_score` variable should be audited for potential target leakage.

3.5 Research Gap and Project Contribution

The practical question is not merely which algorithm obtains the largest accuracy value, but how different model families behave under class imbalance and a consistent evaluation protocol. The project compares baseline models, tuned versions, an ensemble, imbalance-handling variants, and neural networks. It also includes performance visualisations and model interpretation. Its contribution is an empirical comparison on the supplied dataset; generalisation to other businesses or time periods has not yet been established.

3.6 Survey Summary Table

Table 3.1: Selected literature relevant to the project

No.	Reference	Problem addressed	Approach / contribution	Relevance
1	Breiman (2001) [1]	General classification and regression	Introduces Random Forests using bagged trees and random feature selection	Basis for Random Forest baseline
2	Chen and Guestrin (2016) [2]	Efficient boosted-tree learning	Presents XGBoost system design and regularised boosting	Basis for XGBoost models
3	Prokhorenkova et al. (2018) [3]	Gradient boosting with categorical data	Introduces ordered boosting and categorical processing in CatBoost	Relevant to CatBoost comparison

No.	Reference	Problem addressed	Approach / contribution	Relevance
4	Chawla et al. (2002) [4]	Minority-class underrepresentation	Introduces SMOTE synthetic minority oversampling	Basis for resampling variants
5	Lundberg and Lee (2017) [5]	Explaining complex model predictions	Defines SHAP as a unified feature-attribution framework	Basis for model interpretation
6	Pedregosa et al. (2011) [6]	Reproducible machine-learning workflows	Describes scikit-learn estimator, preprocessing, and evaluation tools	Supports experiment implementation
7	Salton and Buckley (1988) [7]	Term weighting in information retrieval	Studies term-weighting approaches including TF-IDF-style representations	Context for text-derived feature engineering
8	Grinsztajn et al. (2022) [8]	Tabular learning benchmarks	Analyses why tree-based methods remain competitive on tabular data	Context for ML versus neural network comparison

The table summarises foundational methodological papers rather than claiming that each paper studies real-estate leads specifically. Domain-specific literature should be added if required by the supervisor.

CHAPTER 4 SYSTEM REQUIREMENT SPECIFICATION

4.1 Functional Requirements

1. Load the preprocessed dataset and validate the presence of the target and input columns.
2. Exclude the identifier, split indicator, and target from the predictor matrix.
3. Preserve the supplied stratified split where available, or create a stratified 80:20 split.
4. Train baseline machine-learning classifiers and tuned variants.
5. Evaluate the soft-voting ensemble and class-imbalance variants.
6. Train and validate the MLP, Deep MLP, and Residual MLP models.
7. Compute accuracy, precision, recall, F1-score, ROC-AUC, and confusion matrices.
8. Generate comparative plots, feature-importance charts, and SHAP summaries.
9. Save model artifacts and verify that reloaded models reproduce predictions.

4.2 Non-functional Requirements

- **Reproducibility:** random seeds and split logic should be recorded.
- **Reliability:** preprocessing and evaluation should be repeatable without accidental target leakage.
- **Maintainability:** preprocessing, training, evaluation, and interpretation should remain modular.
- **Scalability:** the workflow should support retraining when additional labelled leads become available.
- **Interpretability:** metrics and explanation plots should be available to help review model behaviour.
- **Privacy:** lead data should be accessed and stored according to the organisation's data-protection requirements.

4.3 Hardware Requirements

The experiments require a computer capable of running Python and holding the dataset and encoded feature matrix in memory. Exact runtime and hardware requirements depend on model settings and library versions. A multi-core CPU is useful for tree ensembles and cross-validation. A GPU is optional for the neural-network experiments; the supplied notebooks do not establish that a GPU is required.

4.4 Software Requirements

Table 4.1: Software components used or indicated by the notebooks

Component	**Purpose**
Python and Jupyter Notebook	Experiment implementation and execution
pandas, NumPy	Tabular data handling and numerical operations
scikit-learn	Splitting, preprocessing utilities, classifiers, metrics, and cross-validation
XGBoost, LightGBM, CatBoost	Gradient-boosted tree classifiers
TensorFlow / Keras	Neural-network architectures and training
imbalanced-learn	SMOTE and imbalanced-learning pipeline support
SHAP	Feature-attribution explanations
Matplotlib / Seaborn	Result visualisation
joblib	Saving and reloading conventional ML models

Exact library versions should be recorded from the environment before final submission.

CHAPTER 5 SYSTEM DESIGN

5.1 System Architecture

The system can be organised into data ingestion, preprocessing, model training, model selection, evaluation, and interpretation modules. The supplied preprocessing notebook creates an encoded dataset, while the classification notebook loads the encoded numeric matrix and performs model experiments. Separating preprocessing from model training is convenient, but it creates a requirement to preserve fold-specific transformations during cross-validation, particularly for target encoding.

5.2 Module Design

Data module

Loads the processed dataset and checks target, split, and feature columns.

Preprocessing module

Applies encoding and numerical transformations to raw lead fields.

Training module

Fits baseline, tuned, ensemble, imbalance-aware, and neural-network models.

Evaluation module

Calculates metrics from test labels and predictions and produces comparison tables.

Interpretation module

Generates feature importance and SHAP plots.

Persistence module

Saves selected models and checks prediction consistency after reloading.

5.3 Detailed Design

5.3.1 Data Preprocessing

The preprocessing notebook represents a mixed-feature transformation workflow. The documented pipeline includes one-hot encoding, ordinal encoding, smoothed out-of-fold target encoding for preferred location, frequency encoding, TF-IDF features for `item_checked` sequences, a demonstration-request count and indicator, a logarithmic transformation of budget

using $\log(1+x)$, and standard scaling. The classification notebook expects a fully numeric processed matrix. The final report should be checked against the actual preprocessing notebook and its output columns to ensure these transformations and their ordering match the version submitted.

5.3.2 Class Diagram

Figure 5.1 is a conceptual component diagram, not a UML class diagram generated from application source code.

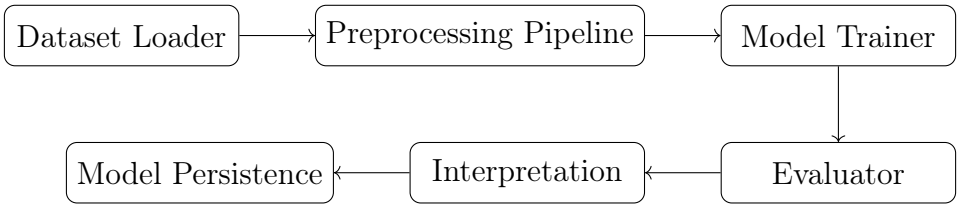


Figure 5.1: Conceptual component diagram for the experimental workflow.

5.3.3 Activity Diagram

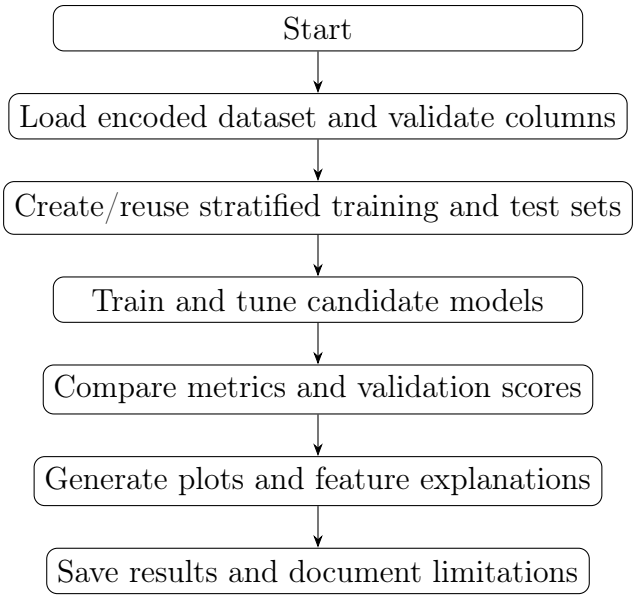


Figure 5.2: Activity flow for model comparison.

5.3.4 Use Case Diagram

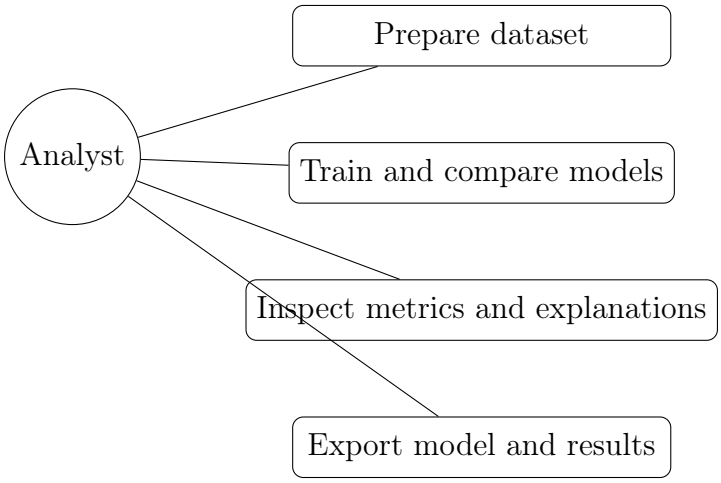


Figure 5.3: Use-case view of the research workflow.

5.3.5 Scenarios

Scenario 1: Model comparison. The analyst loads the encoded dataset, confirms the target and split fields, trains candidate models, and reviews the metrics table. The analyst selects a candidate using a training-side score and then reports its test-set metrics.

Scenario 2: Minority-class emphasis. The analyst compares the original classifier against class-weighted and SMOTE variants. The analyst reviews recall and precision together to decide whether the increase in detected converters justifies the additional false positives.

Scenario 3: Model interpretation. The analyst reviews tree feature-importance plots and SHAP summary plots. Any feature with suspiciously high importance is checked for timing and data provenance before it is used in a deployed model.

CHAPTER 6 IMPLEMENTATION

6.1 Development Environment and Input

The implementation is contained in two Jupyter notebooks: one for preprocessing and one for classification. The classification notebook searches for the encoded dataset in common local or notebook directories, validates that the features are numeric, and reuses the stored train/test split when it is present. It excludes the identifier and bookkeeping fields before fitting the models. For reproducibility, the final project submission should include the notebooks, processed-data schema, dependency list, and instructions for obtaining the dataset without exposing private lead information.

6.2 Preprocessing Implementation

The preprocessing stage converts raw categorical, numerical, and sequence-like fields into a numeric representation. One-hot and ordinal encodings represent categories; target encoding uses outcome statistics and must be calculated without using validation-fold labels; frequency encoding maps values to their occurrence frequencies; TF-IDF represents tokens in item sequences; budget is transformed using $\log(1 + x)$; and standardisation places selected numeric features on comparable scales. Scaling is especially important for distance-based methods and SVMs, while tree-based models are generally less sensitive to monotonic scaling. The final model matrix contains 84 features according to the classification notebook.

6.3 Baseline Machine-Learning Models

Ten initial algorithms are evaluated: Logistic Regression, Decision Tree, Random Forest, Extra Trees, Gradient Boosting, XGBoost, LightGBM, CatBoost, Support Vector Machine, and K-Nearest Neighbours. The notebook records accuracy, precision, recall, F1-score, ROC-AUC, and a training-side selection score. Confusion matrices are generated for the initial models, allowing false positives and false negatives to be compared beyond a single aggregate metric.

6.4 Hyperparameter Tuning

Randomized search is used for selected tree ensembles, including Random Forest, XGBoost, and CatBoost; grid search is used for Logistic Regression and SVM. The notebook uses three-fold stratified cross-validation on training data for model selection. Hyperparameter tuning can improve performance, but the estimate is only trustworthy if all label-dependent preprocessing is refitted inside each fold. Because the supplied preprocessing notebook includes target encoding

and the classification notebook cross-validates the already encoded data, this is a specific issue that must be audited before interpreting cross-validation scores as unbiased.

6.5 Soft-Voting Ensemble

The ensemble combines tuned Logistic Regression, Random Forest, XGBoost, and CatBoost using soft voting. Each classifier contributes class probabilities, which are aggregated to obtain the ensemble prediction. Soft voting can reduce dependence on a single decision boundary when component models make complementary errors. In the reported results, the ensemble obtains the highest test accuracy, but the improvement over tuned XGBoost is small and has not been shown to be statistically significant.

6.6 Imbalance Handling

The positive class is approximately 27% of the dataset. The notebook compares tuned Logistic Regression and tuned XGBoost with no imbalance adjustment, class weighting, and SMOTE. SMOTE should be used within training folds only so that synthetic examples cannot be generated using information from held-out observations. The imbalance-aware variants increase recall substantially but lower accuracy and precision, which is consistent with a more aggressive prediction of the positive class.

6.7 Deep-Learning Models

Three neural-network architectures are evaluated: MLP, Deep MLP, and Residual MLP. The data is converted to 32-bit floating-point arrays for neural-network training. A validation partition is taken from the training data, and training/validation accuracy and loss are recorded across epochs. The best neural network by validation accuracy is the Residual MLP, with validation accuracy 0.7757 and test accuracy 0.7700 according to the notebook results. The neural network is not selected solely by its test accuracy.

6.8 Model Saving and Reproducibility

The notebook saves the best conventional model using joblib and the best neural network in Keras format, then reloads the saved models to check prediction consistency. For a fully reproducible submission, record Python and library versions, random seeds, exact input file names, the feature list and order, and the model artefact hashes. Any model intended for operational scoring must package the preprocessing transformations with the estimator so that new records are transformed identically.

CHAPTER 7 RESULTS

7.1 Evaluation Metrics

Let TP, TN, FP, and FN denote true positives, true negatives, false positives, and false negatives. Accuracy measures the fraction of correct predictions:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}.$$

Precision measures the fraction of predicted positives that are correct:

$$\text{Precision} = \frac{TP}{TP + FP}.$$

Recall measures the fraction of actual positives identified:

$$\text{Recall} = \frac{TP}{TP + FN}.$$

The F1-score is the harmonic mean of precision and recall:

$$F_1 = 2\frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

ROC-AUC measures ranking performance across decision thresholds. Because the data is imbalanced, the metrics are interpreted together rather than using accuracy alone.

7.2 Class Distribution

The target distribution figure from the notebook shows approximately 73% non-converted and 27% converted leads. This imbalance explains why the majority-class baseline is already close to 0.73 accuracy. A model should therefore be evaluated for its ability to identify positive cases and not simply for its total number of correct predictions.

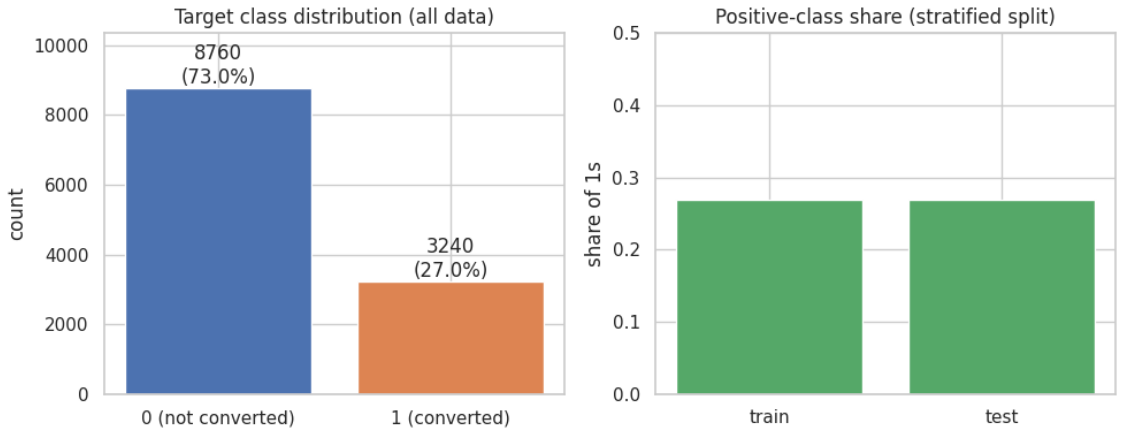


Figure 7.1: Target class distribution and positive-class share across the stratified split (notebook output).

7.3 Baseline Confusion Matrices

Figure 7.2 reproduces the notebook's confusion-matrix grid for the initial models. The matrices make the class trade-offs visible. A model may have similar accuracy to another while missing a different number of converted leads. In lead prioritisation, false negatives represent potential converters not flagged by the model, while false positives represent leads flagged for follow-up that do not convert.

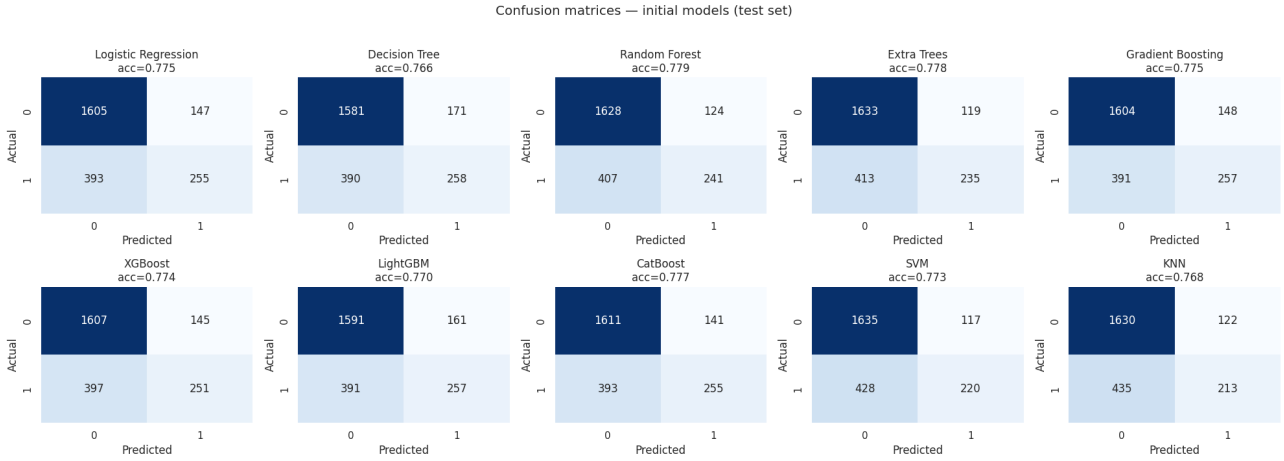


Figure 7.2: Confusion matrices for the initial machine-learning models on the test set.

7.4 Comparison Across Model Families

The full notebook evaluates 24 model configurations, including original models, tuned variants, the voting ensemble, imbalance-aware variants, and neural networks. The top test accuracy is 0.7792 for the tuned voting ensemble. The tuned XGBoost model selected by the training-side selection score achieves 0.7762 test accuracy. The difference in accuracy is 0.0030, or 0.30 percentage points. The notebook does not provide confidence intervals or a statistical

significance test for this difference, so it should not be described as a proven performance advantage.

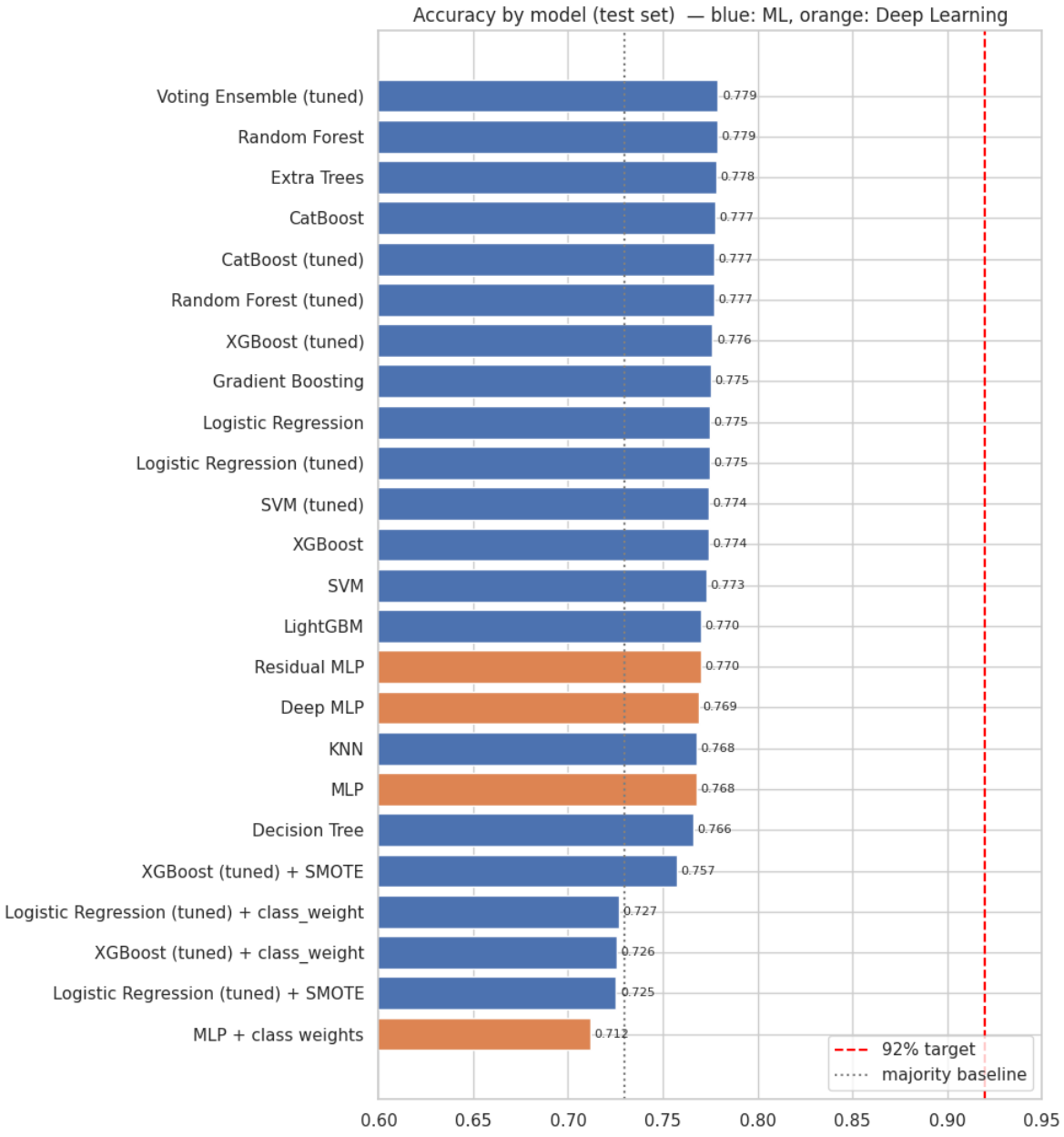


Figure 7.3: Model comparison visualisation from the notebook.

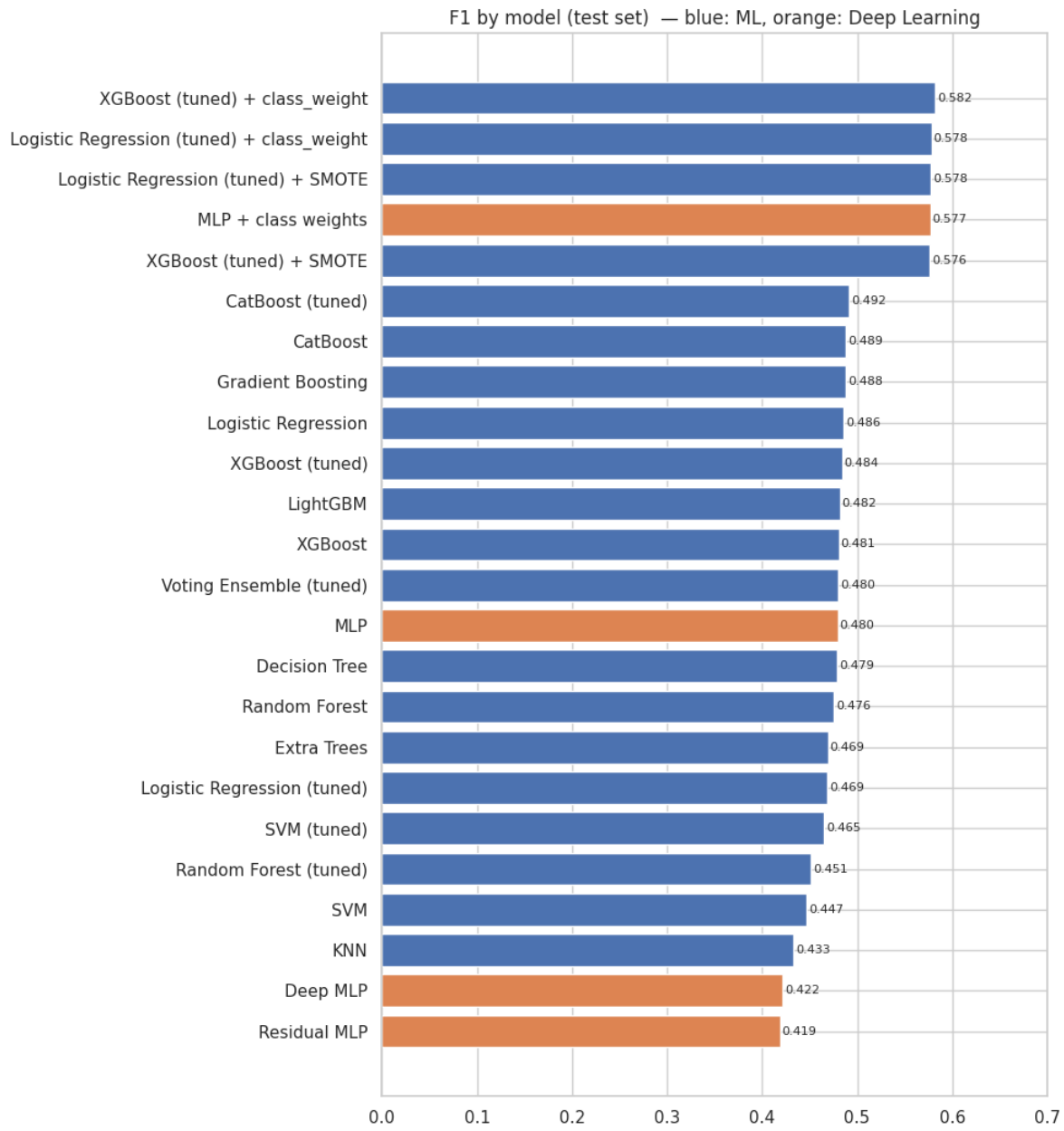


Figure 7.4: Additional model comparison visualisation from the notebook.

7.5 Summary of All 24 Configurations

Table 7.1 records the test metrics reported by the classification notebook. Values are rounded to four decimal places. The selection score is the training-side cross-validation score for conventional ML configurations and the validation accuracy for deep-learning configurations, as represented by the notebook.

7.6 Selected Conventional Model

Tuned XGBoost is selected by the highest training-side selection score reported in the notebook (0.7833, tied at the rounded value with the ensemble). Its test accuracy is 0.7762, precision

Table 7.1: Complete comparison of evaluated model configurations

No.	Model	Accuracy	Precision	Recall	F1	ROC-AUC	Selection
1	Voting Ensemble (tuned)	0.7792	0.6586	0.3781	0.4804	0.7823	0.7833
2	Random Forest	0.7788	0.6603	0.3719	0.4758	0.7706	0.7761
3	Extra Trees	0.7783	0.6638	0.3627	0.4691	0.7731	0.7753
4	CatBoost	0.7775	0.6439	0.3935	0.4885	0.7756	0.7775
5	Random Forest (tuned)	0.7771	0.6728	0.3395	0.4513	0.7816	0.7810
6	CatBoost (tuned)	0.7771	0.6395	0.3997	0.4919	0.7817	0.7823
7	XGBoost (tuned; selected)	0.7762	0.6412	0.3889	0.4841	0.7807	0.7833
8	Gradient Boosting	0.7754	0.6346	0.3966	0.4881	0.7807	0.7763
9	Logistic Regression (tuned)	0.7750	0.6467	0.3673	0.4685	0.7808	0.7830
10	Logistic Regression	0.7750	0.6343	0.3935	0.4857	0.7804	0.7806
11	XGBoost	0.7742	0.6338	0.3873	0.4808	0.7754	0.7761
12	SVM (tuned)	0.7742	0.6448	0.3642	0.4655	0.7772	0.7806
13	SVM	0.7729	0.6528	0.3395	0.4467	0.7454	0.7797
14	LightGBM	0.7700	0.6148	0.3966	0.4822	0.7681	0.7691
15	Residual MLP	0.7700	0.6589	0.3071	0.4189	0.7718	0.7757
16	Deep MLP	0.7692	0.6516	0.3117	0.4217	0.7725	0.7750
17	MLP	0.7679	0.6076	0.3966	0.4799	0.7781	0.7653
18	KNN	0.7679	0.6358	0.3287	0.4334	0.7637	0.7758
19	Decision Tree	0.7662	0.6014	0.3981	0.4791	0.7569	0.7590
20	XGBoost (tuned) + SMOTE	0.7575	0.5455	0.6111	0.5764	0.7788	0.7555
21	Logistic Regression (tuned) + class weights	0.7267	0.4956	0.6944	0.5784	0.7810	0.7305
22	XGBoost (tuned) + class weights	0.7258	0.4946	0.7083	0.5825	0.7799	0.7258
23	Logistic Regression (tuned) + SMOTE	0.7254	0.4940	0.6960	0.5778	0.7792	0.7301
24	MLP + class weights	0.7117	0.4777	0.7284	0.5770	0.7789	0.7042

0.6412, recall 0.3889, F1-score 0.4841, and ROC-AUC 0.7807. This profile indicates that its positive predictions are moderately precise but it identifies fewer than half of the actual positive cases at the selected threshold. Whether this is acceptable depends on the operational objective and follow-up capacity.

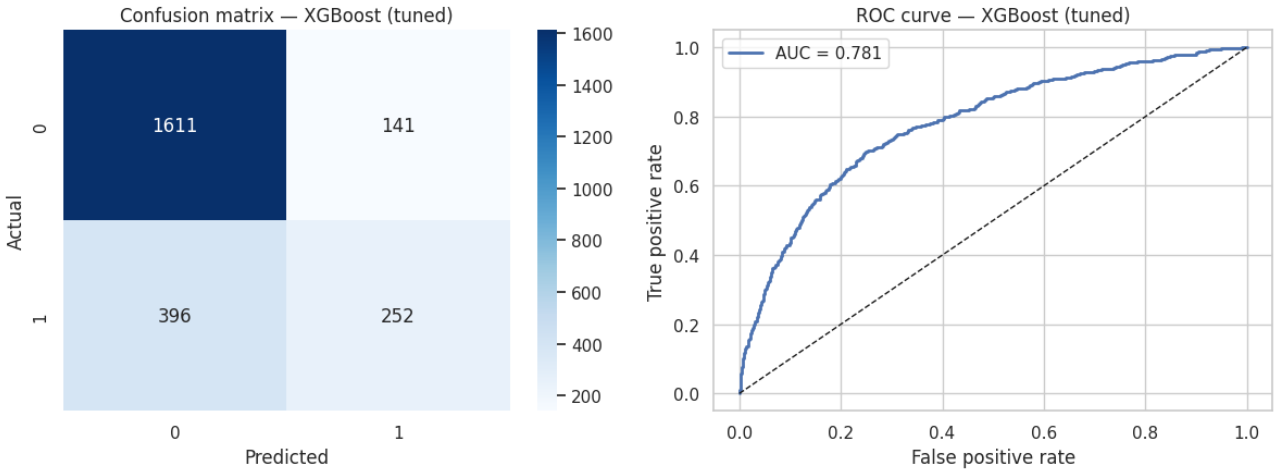


Figure 7.5: Selected-model confusion matrix and ROC curve from the notebook.

7.7 Neural Network Results

Among the evaluated neural networks, Residual MLP has the highest validation selection accuracy of 0.7757 and test accuracy of 0.7700. Deep MLP reports test accuracy 0.7692, and MLP reports 0.7679. These values are below the highest test accuracy of the soft-voting ensemble. The comparison does not imply that neural networks are inherently unsuitable; it indicates that, for this encoded dataset and experimental configuration, the neural networks did not outperform the leading ensemble on test accuracy.

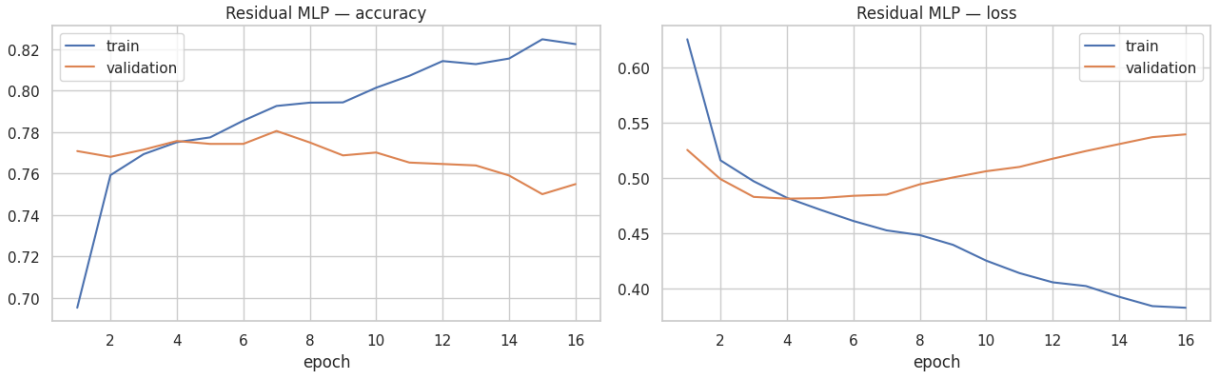


Figure 7.6: Training and validation accuracy/loss curves for the selected neural network.

7.8 Class Weighting and SMOTE

Imbalance-aware configurations achieve higher recall: class-weighted XGBoost reports recall 0.7083, while class-weighted Logistic Regression reports recall 0.6944. XGBoost with SMOTE

reports recall 0.6111. These variants have lower accuracy (approximately 0.71–0.76) and precision (approximately 0.48–0.55) than the highest-accuracy configurations. This trade-off is expected when the model is encouraged to identify more positive cases. A deployment decision should be made using precision-recall curves, calibration, business costs, and a threshold selected on validation data, rather than by accuracy alone.

7.9 Feature Importance

The notebook calculates feature importance for Random Forest (tuned), Extra Trees, XGBoost (tuned), and CatBoost (tuned), and compares average importance across tree models. These plots can identify candidate predictors for further analysis. However, the report should not assign domain meaning to feature names without checking the feature mapping from the preprocessing notebook. Feature importance describes model behaviour on this dataset and does not establish that a feature causes conversion.

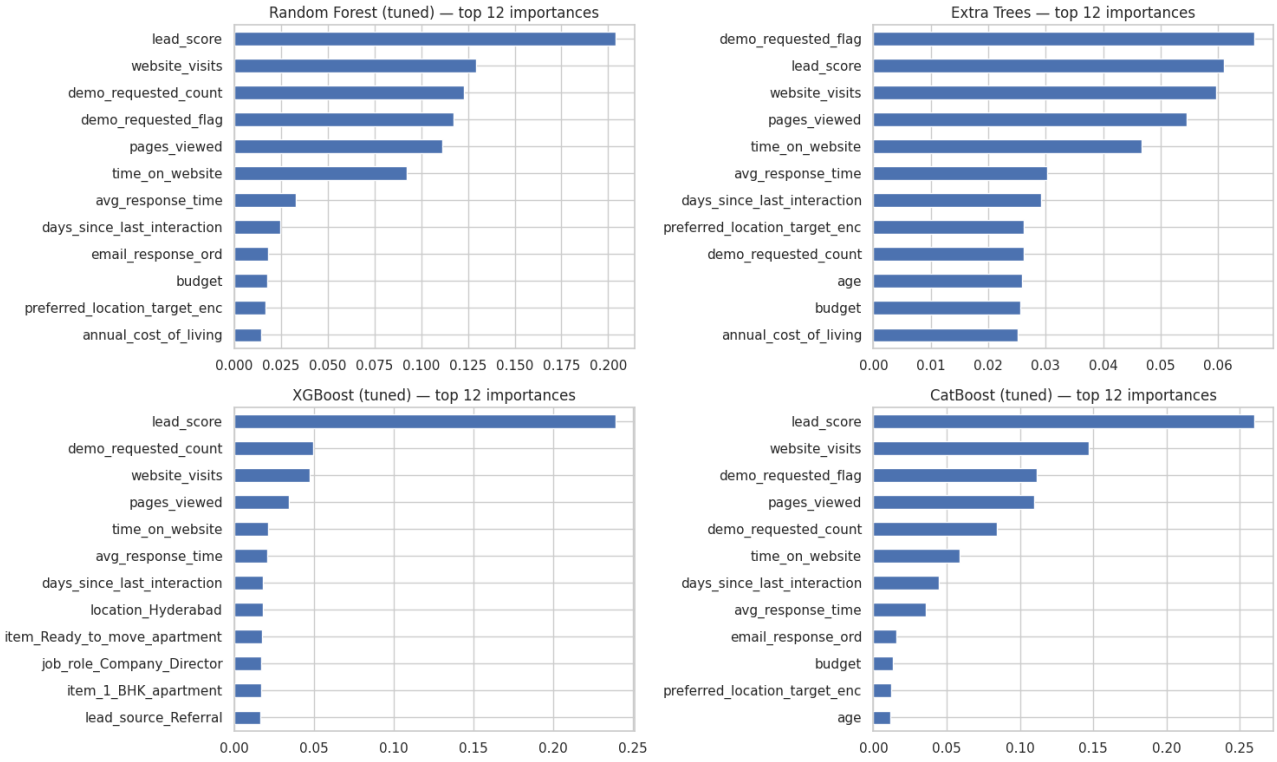


Figure 7.7: Top feature importances across tree-based models, as generated by the notebook.

7.10 SHAP Analysis

SHAP explanations are calculated for a sample of 500 test rows using the selected tree-based model. The bar summary ranks features by mean absolute attribution, while the beeswarm-style summary displays the distribution and direction of feature contributions. The two views complement the model's native feature-importance scores. Correlated features can share or

redistribute attribution, and explanations should be treated as descriptions of the fitted model rather than causal effects.

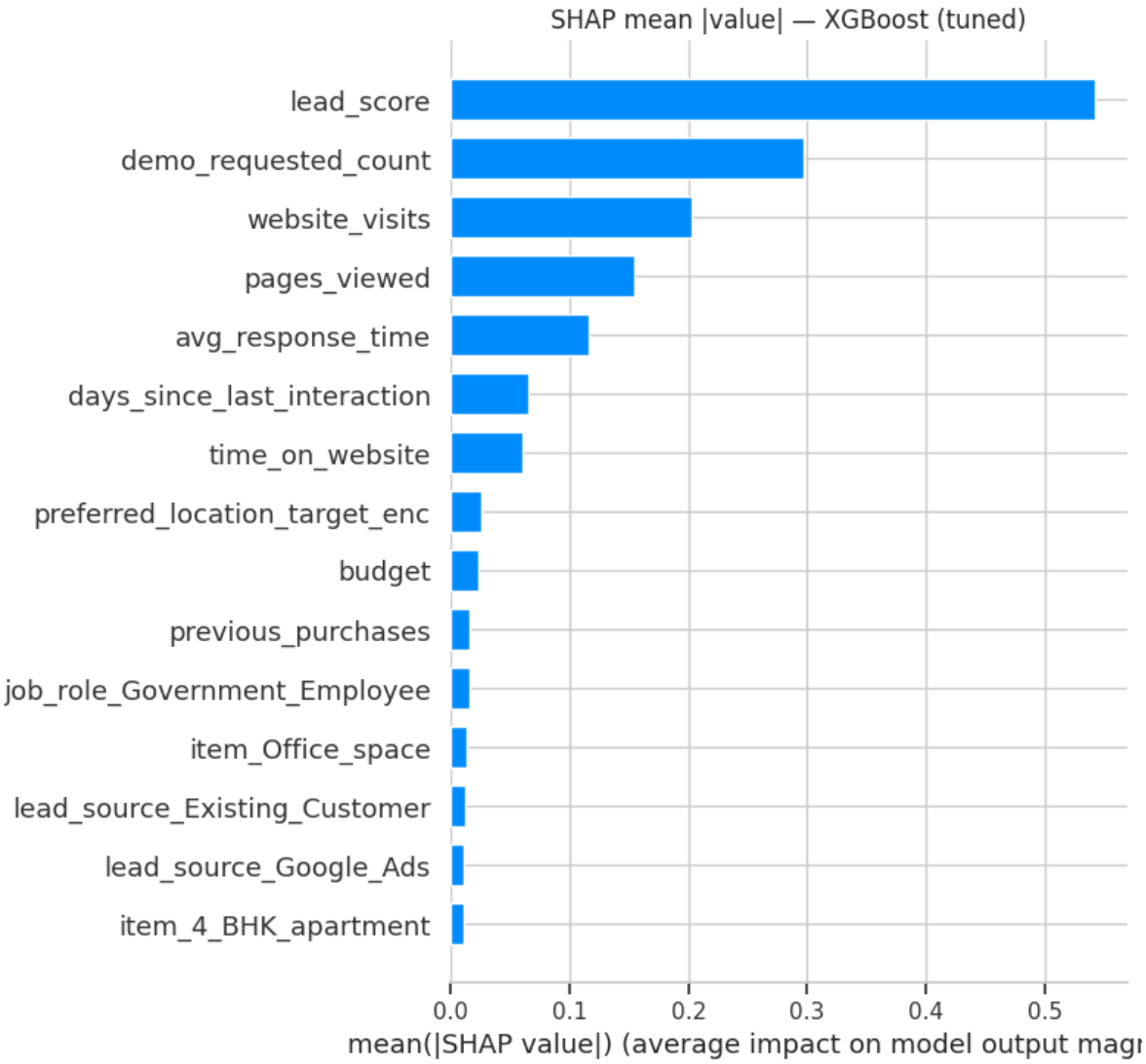


Figure 7.8: SHAP global feature-attribution plots from the notebook.



Figure 7.9: Second SHAP summary visualisation from the notebook.

7.11 Accuracy Target and Interpretation

The notebook sets an aspirational accuracy target of 92%. The best observed test accuracy is 77.92%, compared with a majority-class baseline of approximately 73%. Thus, the experiment shows a modest absolute improvement over the baseline but does not meet the stated target. A 92% target should not be pursued by repeatedly tuning against the test set or by using information unavailable at prediction time. The next step should be to improve the validity of the evaluation, inspect possible leakage, obtain more informative features, and evaluate business-relevant metrics.

CHAPTER 8 TESTING

8.1 Testing Strategy

Testing in this project refers to validation of the data pipeline, training process, evaluation calculations, and saved-model behaviour. The notebook includes data checks, stratified partitioning, cross-validation for tuning, held-out testing, and a reload check for model predictions. The following table lists a report-ready testing plan. Mark the status only after the corresponding check has been executed and evidence saved.

Table 8.1: Testing scenarios and expected outcomes

ID	Test scenario	Procedure	Expected outcome	Status
T01	Input schema	Load processed data and inspect target, split, identifier and feature columns	Required columns exist; target excluded from features	*[Fill in: Verify]*
T02	Numeric feature matrix	Check predictor data types and missing-value handling	Model input is numeric and valid	*[Fill in: Verify]*
T03	Split integrity	Confirm train/test row counts and class proportions	Approximately 80:20 stratified partition	*[Fill in: Verify]*
T04	Identifier leakage	Check `lead_id` and split fields are excluded	No identifier/bookkeeping fields enter the model	*[Fill in: Verify]*
T05	Metric calculation	Recompute metrics from saved predictions	Metrics match the results table	*[Fill in: Verify]*
T06	Cross-validation integrity	Refit every label-dependent transformation inside each fold	No validation-fold labels influence training features	**Required**

ID	Test scenario	Procedure	Expected outcome	Status
T07	SMOTE integrity	Inspect pipeline to confirm resampling happens only on fold training data	Validation and test records remain untouched	*[Fill in: Verify]*
T08	Model persistence	Reload saved model and compare predictions	Predictions match those from the in-memory model	Notebook reports check
T09	Leakage audit	Trace `lead_score` creation and availability time	Feature exists before conversion and contains no target leakage	**Required**
T10	Robustness	Evaluate on a later time period or external dataset	Performance is stable enough for intended use	Future work

8.2 Important Validation Risks

8.2.1 Target-Encoding Cross-Validation

The preprocessing notebook describes smoothed out-of-fold target encoding, and the classification notebook then performs three-fold cross-validation on the encoded matrix. If encodings for rows in an outer validation fold were calculated using labels from that same fold, the cross-validation score can be optimistic. To correct this, target encoding and all other learned preprocessing must be fitted inside each outer training fold, then applied to that fold's validation rows. The held-out test set should remain untouched until final evaluation.

8.2.2 Potential Target Leakage in `lead_score`

The feature `lead_score` requires a provenance audit. If it is generated using conversion status, post-conversion behaviour, or information that would not exist at the time a prediction is made, its inclusion leaks target or future information. The model should be rerun without this feature and the results compared. The report does not assume that leakage is present; it identifies the issue as a necessary check.

8.2.3 Test-Set Reuse

Repeatedly comparing models on the same test set can create informal selection bias even if formal tuning uses training-only cross-validation. A final locked test set or later time-based holdout should be used once model development is complete. Confidence intervals, repeated seeds, and paired model comparisons would provide stronger evidence than small point-estimate differences.

CHAPTER 9 CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

This project compares a broad set of conventional machine-learning and neural-network approaches for predicting lead conversion in a real-estate and construction lead dataset. The pipeline covers preprocessing, ten initial machine-learning models, hyperparameter tuning, a soft-voting ensemble, class-weighting and SMOTE experiments, and three neural-network architectures. The evaluation includes accuracy, precision, recall, F1-score, ROC-AUC, confusion matrices, learning curves, tree feature importance, and SHAP explanations.

The tuned soft-voting ensemble obtains the highest reported test accuracy of 0.7792. Tuned XGBoost is selected based on the training-side selection score and achieves test accuracy 0.7762 with ROC-AUC 0.7807. The best neural-network validation configuration is Residual MLP, with validation accuracy 0.7757 and test accuracy 0.7700. Class-weighted models improve recall but reduce accuracy and precision, showing that the most appropriate model depends on the intended balance between missed conversions and unnecessary follow-up. None of the evaluated configurations reaches the aspirational 92% accuracy target. The reported improvement over the majority-class baseline is modest, and the results should be considered preliminary until validation concerns are resolved.

9.2 Future Enhancements

1. **Leakage-safe pipeline:** integrate all preprocessing, especially target encoding, inside cross-validation folds.

2. **Audit `lead_score`:** verify that it is available before conversion and does not encode the label; run a feature-ablation experiment.

3. **Threshold optimisation:** select an operating threshold on validation data using precision-recall trade-offs and sales-team capacity.

4. **Additional metrics:** report PR-AUC, balanced accuracy, calibration, and decision-curve or cost-based metrics.

5. **Robustness:** use repeated stratified cross-validation and confidence intervals for performance estimates.

6. **Temporal evaluation:** test on leads collected after the training period to simulate real deployment.

7. **Data improvement:** verify labels and investigate additional pre-conversion features with valid provenance.

8. **External validation:** test on another time period, project, region, or organisation before generalising conclusions.

9. **Operational integration:** package preprocessing and inference, monitor drift, and retrain under a documented governance process.

10. **Fairness and privacy:** review whether features create inappropriate disparate outcomes and follow applicable data-protection requirements.

Bibliography

[1] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, pp. 5–32, 2001.

[2] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[3] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: unbiased boosting with categorical features," in *Advances in Neural Information Processing Systems*, vol. 31, 2018.

[4] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[5] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[6] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[7] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.

[8] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?" in *Advances in Neural Information Processing Systems*, vol. 35, 2022.

[9] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York, NY, USA: Springer, 2006.

[10] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, 2nd ed. New York, NY, USA: Springer, 2009.

[11] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.

[12] J. Davis and M. Goadrich, "The relationship between Precision-Recall and ROC curves," in *Proceedings of the 23rd International Conference on Machine Learning*, 2006, pp. 233–240.